Tutorial: Use the ComWrappers API

Tutorial: Use the `ComWrappers` API

In this tutorial, you'll learn how to properly subclass the `ComWrappers` type to provide an optimized and AOT-friendly COM interop solution. Before starting this tutorial, you should be familiar with COM, its architecture, and existing COM interop solutions.

In this tutorial, you'll implement the following interface definitions. These interfaces and their implementations will demonstrate:

- Marshalling and unmarshalling types across the COM/.NET boundary.
- Two distinct approaches to consuming native COM objects in .NET.
- A recommended pattern for enabling custom COM interop in .NET 5 and beyond.

All source code used in this tutorial is available in the dotnet/samples repository.

Note

In the .NET 8 SDK and later versions, a source generator is provided to automatically generate a `ComWrappers` API implementation for you. For more information, see `ComWrappers` source generation.

C# definitions

csharp

```csharp
interface IDemoGetType
{
    string? GetString();
}

interface IDemoStoreType
{
    void StoreString(int len, string? str);
}
```

Win32 C++ definitions

cpp

```cpp
MIDL_INTERFACE("92BAA992-DB5A-4ADD-977B-B22838EE91FD")
IDemoGetType : public IUnknown
{
    HRESULT STDMETHODCALLTYPE GetString(_Outptr_ wchar_t** str) = 0;
};

MIDL_INTERFACE("30619FEA-E995-41EA-8C8B-9A610D32ADCB")
IDemoStoreType : public IUnknown
{
    HRESULT STDMETHODCALLTYPE StoreString(int len, _In_z_ const wchar_t* str) = 0;
};
```

The `ComWrappers` API was designed to provide the minimal interaction needed to accomplish COM interop with the .NET 5+ runtime. This means that many of the niceties that exist with the built-in COM interop system are not present and must be built up from basic building blocks. The two primary responsibilities of the API are:

- Efficient object identification (for example, mapping between an `IUnknown*` instance and a managed object).
- Garbage Collector (GC) interaction.

These efficiencies are accomplished by requiring wrapper creation and acquisition to go through the `ComWrappers` API.

Since the `ComWrappers` API has so few responsibilities, it stands to reason that most of the interop work should be handled by the consumer – this is true. However, the additional work is largely mechanical and can be performed by a source-generation solution. As an example, the C#/WinRT tool chain is a source-generation solution that's built on top of `ComWrappers` to provide WinRT interop support.

Providing a `ComWrappers` subclass means giving enough information to the .NET runtime to create and record wrappers for managed objects being projected into COM and COM objects being projected into .NET. Before we look at an outline of the subclass, we should define some terms.

Managed Object Wrapper – Managed .NET objects require wrappers to enable usage from a non-.NET environment. These wrappers are historically called COM Callable Wrappers (**CCW**).

Native Object Wrapper – COM objects that are implemented in a non-.NET language require wrappers to enable usage from .NET. These wrappers are historically called Runtime Callable Wrappers (**RCW**).

Step 1 – Define methods to implement and understand their intent

To extend the `ComWrappers` type, you must implement the following three methods. Each of these methods represents the user's participation in the creation or deletion of a type of wrapper. The `ComputeVtables()` and `CreateObject()` methods create a Managed Object Wrapper and Native Object Wrapper, respectively. The `ReleaseObjects()` method is used by the runtime to make a request for the supplied collection of wrappers to be "released" from the underlying native object. In most cases, the body of the `ReleaseObjects()` method can simply throw NotImplementedException, since it's only called in an advanced scenario involving the Reference Tracker framework.

csharp
```csharp
// See referenced sample for implementation.
class DemoComWrappers : ComWrappers
{
    protected override unsafe ComInterfaceEntry* ComputeVtables(object obj, CreateComInterfaceFlags flags, out int count) =>
        throw new NotImplementedException();

    protected override object? CreateObject(IntPtr externalComObject, CreateObjectFlags flags) =>
        throw new NotImplementedException();

    protected override void ReleaseObjects(IEnumerable objects) =>
        throw new NotImplementedException();
}
```

To implement the `ComputeVtables()` method, decide which managed types you'd like to support. For this tutorial, we'll support the two previously defined interfaces (`IDemoGetType` and `IDemoStoreType`) and a managed type that implements the two interfaces (`DemoImpl`).

csharp
```csharp
class DemoImpl : IDemoGetType, IDemoStoreType
{
    string? _string;
    public string? GetString() => _string;
    public void StoreString(int _, string? str) => _string = str;
}
```

For the `CreateObject()` method, you'll also need to determine what you'd like to support. In this case, though, we only know the COM interfaces we're interested in, not the COM classes. The interfaces being consumed from the COM side are the same as the ones we're projecting from the .NET side (that is, `IDemoGetType` and `IDemoStoreType`).

We won't implement `ReleaseObjects()` in this tutorial.

Let's start with the Managed Object Wrapper – these wrappers are easier. You'll build a Virtual Method Table, or *vtable*, for each interface in order to project them into the COM environment. For this tutorial, you'll define a vtable as a sequence of pointers, where each pointer represents an implementation of a function on an interface – order is *very* important here. In COM, every interface inherits from `IUnknown`. The `IUnknown` type has three methods defined in the following order: `QueryInterface()`, `AddRef()`, and `Release()`. After the `IUnknown` methods come the specific interface methods. For example, consider `IDemoGetType` and `IDemoStoreType`. Conceptually, the vtables for the types would look like the following:

```
IDemoGetType     | IDemoStoreType
================================
QueryInterface   | QueryInterface
AddRef           | AddRef
Release          | Release
GetString        | StoreString
```

Looking at `DemoImpl`, we already have an implementation for `GetString()` and `StoreString()`, but what about the `IUnknown` functions? How to implement an `IUnknown` instance is beyond the scope of this tutorial, but it can be done manually in `ComWrappers`. However, in this tutorial, you'll let the runtime handle that part. You can get the `IUnknown` implementation using the `ComWrappers.GetIUnknownImpl()` method.

It might seem like you've implemented all the methods, but unfortunately, only the `IUnknown` functions are consumable in a COM vtable. Since COM is outside of the runtime, you'll need to create native function pointers to your `DemoImpl` implementation. This can be done using C# function pointers and the `UnmanagedCallersOnlyAttribute`. You can create a function to insert into the vtable by creating a `static` function that mimics the COM function signature. Following is an example of the COM signature for `IDemoGetType.GetString()` – recall from the COM ABI that the first argument is the instance itself.

csharp
```csharp
[UnmanagedCallersOnly]
public static int GetString(IntPtr _this, IntPtr* str);
```

The wrapper implementation of `IDemoGetType.GetString()` should consist of marshalling logic and then a dispatch to the managed object being wrapped. All the state for dispatch is contained within the provided `_this` argument. The `_this` argument will actually be of type `ComInterfaceDispatch*`. This type represents a low-level structure with a single field, `Vtable`, that will be discussed later. Further details of this type and its layout are an implementation detail of the runtime and should not be depended upon. In order to retrieve the managed instance from a `ComInterfaceDispatch*` instance, use the following code:

csharp
```csharp
IDemoGetType inst = ComInterfaceDispatch.GetInstance<IDemoGetType>((ComInterfaceDispatch*)_this);
```

Now that you have a C# method that can be inserted into a vtable, you can construct the vtable. Note the use of `RuntimeHelpers.AllocateTypeAssociatedMemory()` for allocating memory in a way that works with unloadable assemblies.

csharp
```csharp
GetIUnknownImpl(
    out IntPtr fpQueryInterface,
    out IntPtr fpAddRef,
```

```
    out IntPtr fpRelease);

// Local variables with increment act as a guard against incorrect construction of
// the native vtable. It also enables a quick validation of final size.
int tableCount = 4;
int idx = 0;
var vtable = (IntPtr*)RuntimeHelpers.AllocateTypeAssociatedMemory(
    typeof(DemoComWrappers),
    IntPtr.Size * tableCount);
vtable[idx++] = fpQueryInterface;
vtable[idx++] = fpAddRef;
vtable[idx++] = fpRelease;
vtable[idx++] = (IntPtr)(delegate* unmanaged<IntPtr, IntPtr*, int>)&ABI.IDemoGetTypeManagedWrapper.GetString;
Debug.Assert(tableCount == idx);
s_IDemoGetTypeVTable = (IntPtr)vtable;
```

The allocation of vtables is the first part of implementing `ComputeVtables()`. You should also construct comprehensive COM definitions for types that you're planning to support – think `DemoImpl` and what parts of it should be usable from COM. Using the constructed vtables, you can now create a series of `ComInterfaceEntry` instances that represent the complete view of the managed object in COM.

csharp
```
s_DemoImplDefinitionLen = 2;
int idx = 0;
var entries = (ComInterfaceEntry*)RuntimeHelpers.AllocateTypeAssociatedMemory(
    typeof(DemoComWrappers),
    sizeof(ComInterfaceEntry) * s_DemoImplDefinitionLen);
entries[idx].IID = IDemoGetType.IID_IDemoGetType;
entries[idx++].Vtable = s_IDemoGetTypeVTable;
entries[idx].IID = IDemoStoreType.IID_IDemoStoreType;
entries[idx++].Vtable = s_IDemoStoreVTable;
Debug.Assert(s_DemoImplDefinitionLen == idx);
s_DemoImplDefinition = entries;
```

The allocation of vtables and entries for the Managed Object Wrapper can and should be done ahead of time since the data can be used for all instances of the type. The work here could be performed in a `static` constructor or a module initializer, but it should be done ahead of time so the `ComputeVtables()` method is as simple and quick as possible.

csharp
```
protected override unsafe ComInterfaceEntry* ComputeVtables(object obj, CreateComInterfaceFlags flags,
out int count)
{
    if (obj is DemoImpl)
    {
        count = s_DemoImplDefinitionLen;
        return s_DemoImplDefinition;
    }

    // Unknown type
    count = 0;
    return null;
}
```

Once you've implemented the `ComputeVtables()` method, the `ComWrappers` subclass will be able to produce Managed Object Wrappers for instances of `DemoImpl`. Be aware that the returned Managed Object Wrapper from the call to `GetOrCreateComInterfaceForObject()` is of type `IUnknown*`. If the native API that's being passed to the wrapper requires a different interface, a `Marshal.QueryInterface()` for that interface must be performed.

csharp
```
var cw = new DemoComWrappers();
var demo = new DemoImpl();
IntPtr ccw = cw.GetOrCreateComInterfaceForObject(demo, CreateComInterfaceFlags.None);
```

Constructing a Native Object Wrapper has more implementation options and a great deal more nuance than constructing a Managed Object Wrapper. The first question to address is how permissive the `ComWrappers` subclass will be in supporting COM types. To support all COM types, which is possible, you'll need to write a substantial amount of code or employ some clever uses of `Reflection.Emit`. For this tutorial, you'll only support COM instances that implement both `IDemoGetType` and `IDemoStoreType`. Since you know there is a finite set and have restricted that any supplied COM instance must implement both interfaces, you could provide a single, statically defined wrapper; however, dynamic cases are common enough in COM that we'll explore both options.

Static Native Object Wrapper

Let's look at the static implementation first. The static Native Object Wrapper involves defining a managed type that implements the .NET interfaces and can forward the calls on the managed type to the COM instance. A rough outline of the static wrapper follows.

csharp
```
// See referenced sample for implementation.
class DemoNativeStaticWrapper
    : IDemoGetType
    , IDemoStoreType
{
    public string? GetString() =>
        throw new NotImplementedException();

    public void StoreString(int len, string? str) =>
        throw new NotImplementedException();
}
```

To construct an instance of this class and provide it as a wrapper, you must define some policy. If this type is used as a wrapper, it would seem that since it implements both interfaces, the underlying COM instance should implement both interfaces too. Given that you're adopting this policy, you'll need to confirm this through calls to `Marshal.QueryInterface()` on the COM instance.

csharp
```csharp
int hr = Marshal.QueryInterface(ptr, ref IDemoGetType.IID_IDemoGetType, out IntPtr IDemoGetTypeInst);
if (hr != 0)
{
    return null;
}

hr = Marshal.QueryInterface(ptr, ref IDemoStoreType.IID_IDemoStoreType, out IntPtr IDemoStoreTypeInst);
if (hr != 0)
{
    Marshal.Release(IDemoGetTypeInst);
    return null;
}

return new DemoNativeStaticWrapper()
{
    IDemoGetTypeInst = IDemoGetTypeInst,
    IDemoStoreTypeInst = IDemoStoreTypeInst
};
```

Dynamic Native Object Wrapper

Dynamic wrappers are more flexible because they provide a way for types to be queried at runtime instead of statically. To provide this support, you'll use `IDynamicInterfaceCastable`. Observe that `DemoNativeDynamicWrapper` only implements this interface. The functionality that the interface provides is a chance to determine what type is supported at runtime. The source for this tutorial does a static check during creation but that is simply for code sharing since the check could be deferred until a call is made to `DemoNativeDynamicWrapper.IsInterfaceImplemented()`.

csharp
```csharp
// See referenced sample for implementation.
internal class DemoNativeDynamicWrapper
    : IDynamicInterfaceCastable
{
    public RuntimeTypeHandle GetInterfaceImplementation(RuntimeTypeHandle interfaceType) =>
        throw new NotImplementedException();

    public bool IsInterfaceImplemented(RuntimeTypeHandle interfaceType, bool throwIfNotImplemented) =>
        throw new NotImplementedException();
}
```

Let's look at one of the interfaces that `DemoNativeDynamicWrapper` will dynamically support. The following code provides the implementation of `IDemoStoreType` using the *default interface methods* feature.

csharp
```csharp
[DynamicInterfaceCastableImplementation]
unsafe interface IDemoStoreTypeNativeWrapper : IDemoStoreType
{
    public static void StoreString(IntPtr inst, int len, string? str);

    void IDemoStoreType.StoreString(int len, string? str)
    {
        var inst = ((DemoNativeDynamicWrapper)this).IDemoStoreTypeInst;
        StoreString(inst, len, str);
    }
}
```

There are two important things to note in this example:

1. The `DynamicInterfaceCastableImplementationAttribute` attribute. This attribute is required on any type that is returned from a `IDynamicInterfaceCastable` method. It has the added benefit of making IL trimming easier, which means AOT scenarios are more reliable.
2. The cast to `DemoNativeDynamicWrapper`. This is part of the dynamic nature of `IDynamicInterfaceCastable`. The type that's returned from `IDynamicInterfaceCastable.GetInterfaceImplementation()` is used to "blanket" the type that implements `IDynamicInterfaceCastable`. The gist here is the `this` pointer isn't what it pretends to be because we are permitting a case from `DemoNativeDynamicWrapper` to `IDemoStoreTypeNativeWrapper`.

Forward calls to the COM instance

Regardless of which Native Object Wrapper is used, you need the ability to invoke functions on a COM instance. The implementation of `IDemoStoreTypeNativeWrapper.StoreString()` can serve as an example of employing `unmanaged` C# function pointers.

csharp
```csharp
public static void StoreString(IntPtr inst, int len, string? str)
{
    IntPtr strLocal = Marshal.StringToCoTaskMemUni(str);
    int hr = ((delegate* unmanaged<IntPtr, int, IntPtr, int>)(*(*(void***)inst + 3 /* IDemoStoreType.StoreString slot */)))(inst, len, strLocal);
    if (hr != 0)
    {
        Marshal.FreeCoTaskMem(strLocal);
        Marshal.ThrowExceptionForHR(hr);
    }
}
```

Let's examine the dereferencing of the COM instance to access its vtable implementation. The COM ABI defines that the first pointer of an object is to the type's vtable and, from there, the desired slot can be accessed. Let's assume the address of the COM object is `0x10000`. The first pointer-sized value should be the address of the vtable – in this example `0x20000`. Once you're at the vtable, you look for the fourth slot (index 3 in zero-based indexing) to access the `StoreString()` implementation.

```
COM instance
0x10000  0x20000

VTable for IDemoStoreType
0x20000  <Address of QueryInterface>
0x20008  <Address of AddRef>
0x20010  <Address of Release>
0x20018  <Address of StoreString>
```

Having the function pointer then allows you to dispatch to that member function on that object by passing the object instance as the first parameter. This pattern should look familiar based on the function definitions of the Managed Object Wrapper implementation.

Once the `CreateObject()` method is implemented, the `ComWrappers` subclass will be able to produce Native Object Wrappers for COM instances that implement both `IDemoGetType` and `IDemoStoreType`.

csharp
```csharp
IntPtr iunk = ...; // Get a COM instance from native code.
object rcw = cw.GetOrCreateObjectForComInstance(iunk, CreateObjectFlags.UniqueInstance);
```

Step 4 – Handle Native Object Wrapper lifetime details

The `ComputeVtables()` and `CreateObject()` implementations covered some wrapper lifetime details, but there are further considerations. While this can be a short step, it can also significantly increase the complexity of the `ComWrappers` design.

Unlike the Managed Object Wrapper, which is controlled by calls to its `AddRef()` and `Release()` methods, the lifetime of a Native Object Wrapper is nondeterministically handled by the GC. The question here is, when does the Native Object Wrapper call `Release()` on the `IntPtr` that represents the COM instance? There are two general buckets:

1. The Native Object Wrapper's Finalizer is responsible for calling the COM instance's `Release()` method. This is the only time when it's safe to call this method. At this point, it's been correctly determined by the GC that there are no other references to the Native Object Wrapper in the .NET runtime. There can be complexity here if you're properly supporting COM Apartments; for more information, see the Additional considerations section.

2. The Native Object Wrapper implements `IDisposable` and calls `Release()` in `Dispose()`.

Note

The `IDisposable` pattern should only be supported if, during the `CreateObject()` call, the `CreateObjectFlags.UniqueInstance` flag was passed in. If this requirement is not followed, it's possible for disposed Native Object Wrappers to be reused after being disposed.

You now have a `ComWrappers` subclass that can be tested. To avoid creating a native library that returns a COM instance that implements `IDemoGetType` and `IDemoStoreType`, you'll use the Managed Object Wrapper and treat it as a COM instance – this must be possible in order to pass it COM anyways.

Let's create a Managed Object Wrapper first. Instantiate a `DemoImpl` instance and display its current string state.

csharp
```csharp
var demo = new DemoImpl();

string? value = demo.GetString();
Console.WriteLine($"Initial string: {value ?? "<null>"}");
```

Now you can create an instance of `DemoComWrappers` and a Managed Object Wrapper that you can then pass into a COM environment.

csharp
```csharp
var cw = new DemoComWrappers();

IntPtr ccw = cw.GetOrCreateComInterfaceForObject(demo, CreateComInterfaceFlags.None);
```

Instead of passing the Managed Object Wrapper to a COM environment, pretend you just received this COM instance, so you'll create a Native Object Wrapper for it instead.

csharp
```csharp
var rcw = cw.GetOrCreateObjectForComInstance(ccw, CreateObjectFlags.UniqueInstance);
```

With the Native Object Wrapper, you should be able to cast it to one of the desired interfaces and use it as a normal managed object. You can examine the `DemoImpl` instance and observe the impact of operations on the Native Object Wrapper that's wrapping a Managed Object Wrapper that's in turn wrapping the managed instance.

csharp
```csharp
var getter = (IDemoGetType)rcw;
var store = (IDemoStoreType)rcw;

string msg = "hello world!";
store.StoreString(msg.Length, msg);
```

```
Console.WriteLine($"Setting string through wrapper: {msg}");

value = demo.GetString();
Console.WriteLine($"Get string through managed object: {value}");

msg = msg.ToUpper();
demo.StoreString(msg.Length, msg.ToUpper());
Console.WriteLine($"Setting string through managed object: {msg}");

value = getter.GetString();
Console.WriteLine($"Get string through wrapper: {value}");
```

Since your `ComWrapper` subclass was designed to support `CreateObjectFlags.UniqueInstance`, you can clean up the Native Object Wrapper immediately instead of waiting for a GC to occur.

csharp
```
(rcw as IDisposable)?.Dispose();
```

The creation of COM objects is typically performed via COM Activation – a complex scenario outside the scope of this document. In order to provide a conceptual pattern to follow, we introduce the `CoCreateInstance()` API, used for COM Activation, and illustrate how it can be used with `ComWrappers`.

Assume you have the following C# code in your application. The below example uses `CoCreateInstance()` to activate a COM class and the built-in COM interop system to marshal the COM instance to the appropriate interface. Note the use of `typeof(I).GUID` is limited to an assert and is a case of using reflection which can impact if the code is AOT-friendly.

csharp
```
public static I ActivateClass<I>(Guid clsid, Guid iid)
{
    Debug.Assert(iid == typeof(I).GUID);
    int hr = CoCreateInstance(ref clsid, IntPtr.Zero, /*CLSCTX_INPROC_SERVER*/ 1, ref iid, out object obj);
    if (hr < 0)
    {
        Marshal.ThrowExceptionForHR(hr);
    }
    return (I)obj;
}

[DllImport("Ole32")]
private static extern int CoCreateInstance(
    ref Guid rclsid,
    IntPtr pUnkOuter,
    int dwClsContext,
    ref Guid riid,
    [MarshalAs(UnmanagedType.Interface)] out object ppObj);
```

Converting the above to use `ComWrappers` involves removing the `MarshalAs(UnmanagedType.Interface)` from the `CoCreateInstance()` P/Invoke and performing the marshalling manually.

csharp
```
static ComWrappers s_ComWrappers = ...;

public static I ActivateClass<I>(Guid clsid, Guid iid)
{
    Debug.Assert(iid == typeof(I).GUID);
    int hr = CoCreateInstance(ref clsid, IntPtr.Zero, /*CLSCTX_INPROC_SERVER*/ 1, ref iid, out IntPtr obj);
    if (hr < 0)
    {
        Marshal.ThrowExceptionForHR(hr);
    }
    return (I)s_ComWrappers.GetOrCreateObjectForComInstance(obj, CreateObjectFlags.None);
}

[DllImport("Ole32")]
private static extern int CoCreateInstance(
    ref Guid rclsid,
    IntPtr pUnkOuter,
    int dwClsContext,
    ref Guid riid,
    out IntPtr ppObj);
```

It is also possible to abstract away factory-style functions like `ActivateClass<I>` by including the activation logic in the class constructor for a Native Object Wrapper. The constructor can use the `ComWrappers.GetOrRegisterObjectForComInstance()` API to associate the newly constructed managed object with the activated COM instance.

Additional considerations

Native AOT – Ahead-of-time (AOT) compilation provides improved startup cost as JIT compilation is avoided. Removing the need for JIT compilation is also often required on some platforms. Supporting AOT was a goal of the `ComWrappers` API, but any wrapper implementation must be careful not to inadvertently introduce cases where AOT breaks down, such as using reflection. The `Type.GUID` property is an example of where reflection is used, but in a non-obvious way. The `Type.GUID` property uses reflection to inspect the type's attributes and then potentially the type's name and containing assembly in order to generate its value.

Source generation – Most of the code that's needed for COM interop and a `ComWrappers` implementation can likely be autogenerated by some tooling. Source for both types of wrappers could be generated given the proper COM definitions – for example, Type Library (TLB), IDL, or a Primary Interop Assembly (PIA).

Global registration – Since the `ComWrappers` API was designed as a new phase of COM interop, it needed to have some way to partially integrate with the existing system. There are globally impacting static methods on the `ComWrappers` API that permit registration of a global instance for various support. These methods are designed for `ComWrappers` instances that are expecting to provide comprehensive COM interop support in all cases – akin to the built-in COM interop system.

Reference Tracker support – This support is primary used for WinRT scenarios and represents an advanced scenario. For most `ComWrapper` implementations, either a `CreateComInterfaceFlags.TrackerSupport` or `CreateObjectFlags.TrackerObject` flag should throw a NotSupportedException. If you'd like to enable this support, perhaps on a Windows or even non-Windows platform, it is highly recommended to reference the C#/WinRT tool chain.

Aside from the lifetime, type system, and functional features that are discussed previously, a COM-compliant implementation of `ComWrappers` requires additional considerations. For any implementation that will be used on the Windows platform, there are the following considerations:

- **Apartments** – COM's organizational structure for threading is called "Apartments" and has strict rules that must be followed for stable operations. This tutorial does not implement apartment-aware Native Object Wrappers, but any production-ready implementation should be apartment-aware. To accomplish this, we recommend using the `RoGetAgileReference` API introduced in Windows 8. For versions prior to Windows 8, consider the Global Interface Table.
- **Security** – COM provides a rich security model for class activation and proxied permission.